

pickles, you shouldn't need to pass in any more arguments. So, to order two hamburgers with the standard defaults, you might think your call should look like this:

```
orderBurgers(2) # with ketchup, mustard, and pickles
```

However, in Python, this will trigger an error because there is a mismatch between the number of arguments in the call and the number of parameters specified in the function:

```
TypeError: orderBurgers() missing 3 required positional arguments: 'ketchup',  
'mustard', and 'pickles'
```

Let's see how Python allows us to set up optional parameters that can be given default values if nothing is specified.

Positional and Keyword Parameters

Python has two different types of parameters: positional parameters and keyword parameters. *Positional parameters* are the type that we're already familiar with, where each argument in a call has a matching parameter in the function or method definition.

A *keyword parameter* allows you to specify a default value. You write a keyword parameter as a variable name, an equal sign, and a default value, like this:

```
def someFunction(<keywordParameter>=<default value>):
```

You can have multiple keyword parameters, each with a name and a default value.

A function or method can have both positional parameters and keyword parameters, in which case you must specify all positional parameters *before* any keyword parameters:

```
def someOtherFunction(positionalParam1, positionalParam2, ...  
    <keywordParameter1>=<default value 1>,  
    <keywordParameter2>=<default value 2>, ...):
```
